

这是一个非常敏锐且逻辑性极强的问题！直觉上我们会认为，既然语言是连贯的，猜第三个词(Head 2)肯定得先知道第二个词(Head 1)是什么。

但 **MEDUSA** 的核心黑科技恰恰在于打破了这个直觉：它们是完全并行的，绝对不是串行！

为了让你彻底看清它的内部运作，我为你详细梳理一下：

1. 它们是完全并行的(各算各的)

Medusa Head 2 完全不依赖 Medusa Head 1 猜出的词，它们是在同一时间、同步执行的。

你可以回忆一下课程资料中的那张 MEDUSA 架构图。在模型处理完当前的输入(比如 "What will happen if Medusa meets a llama?")后，Transformer 主干网络会输出一个极其丰富的最终隐藏层特征向量(**Last Hidden State**)。

原始的解码头(LM Head)和所有新增的 Medusa Heads，都是并排插在这个隐藏层特征上的。这意味着在一次前向传播(Forward Pass)中，它们同时接到了同一个信号，然后瞬间同时开工：

- 原始 **Head**: 根据特征，预测第 1 个词($t+1$)。
- **Medusa Head 1**: 根据同一个特征，直接盲猜第 2 个词($t+2$)。
- **Medusa Head 2**: 根据同一个特征，直接盲猜第 3 个词($t+3$)。

2. 为什么 Head 2 可以不依赖 Head 1？

这就涉及大模型强大的“表征能力”了。那个“最终隐藏层特征(**Last Hidden State**)”里，其实已经高度浓缩了当前语境下未来可能走向的信息。

Medusa Head 2 被训练出了一种特殊的能力：“跳级预测”。它不需要明确知道 $t+1$ 和 $t+2$ 的具体单词是啥，它直接从当前的语境特征中，去强行提取“第三个位置大概率应该是个什么词”的信息。

3. 这种“盲猜”的代价与解决方案

正是因为 Head 2 是不看 Head 1 结果直接盲猜的，所以它的预测准确率必然会随着距离的增加而下降。课程资料中提到，如果只取 Top-1 的结果，预测下下个词(Next-Next-Token)的准确率只有大约 60%。

那 **MEDUSA** 是怎么解决这个问题的呢？

它不要求这些头给出唯一的死答案，而是让每个头各自给出几个最高概率的备选项(**Top-k Predictions**)。

- Head 1 给出 3 个可能。
- Head 2 给出 3 个可能。

等这些头并行把备选项都吐出来之后，系统才会在外面把它们像排列组合一样拼起来，形成一棵树状候选结构(**Candidate Tree**)。在这个树里，Head 1 的某个选项，会和 Head 2 的某个选项拼接在一起，形成一条完整的路径。

总结一下：

无论是生成草稿(多个 Head 同步开火)，还是验证草稿(Tree Attention 树状验证)，**MEDUSA** 都在疯狂地榨取 GPU 的并行计算能力。这就是它不用独立小模型，依然能跑得飞快的原因！